

Informe de Pruebas Unitarias

Plataforma Libro de Clases Digital

Spring Boot 3.2.5 · Java 21 · JUnit 5 · Mockito · AssertJ · EmbeddedKafka · JaCoCo

15 Tests Totales · 15 Pasados · 0 Fallidos · 85% Cobertura Lineas · 82% Cobertura Ramas

Alumno: Williams Rivas · w.rivas@duocuc.cl · 16 de junio de 2026

Resumen Ejecutivo

La plataforma Libro de Clases Digital cuenta con **15 pruebas automatizadas** distribuidas en 5 clases de test que cubren los microservicios criticos: autenticacion JWT RS256, gestion administrativa de usuarios, registro de asistencia con logica de eventos Kafka, calificaciones academicas y un test de integracion con broker Kafka + Zookeeper embebido. El **100% de los tests pasa** satisfactoriamente bajo Maven con cobertura de lineas del 85% y de ramas del 82%, superando los umbrales minimos configurados en JaCoCo (80% lineas / 70% ramas).

15

TOTAL TESTS

15

PASADOS

85%

COBERTURA LINEAS

82%

COBERTURA RAMAS

Stack de Pruebas

Herramienta	Version	Rol en las pruebas
JUnit 5 Jupiter	5.10.2	Framework principal: @Test, @BeforeEach, @DisplayName, @ExtendWith
Mockito	5.7.0	Mocks y stubs de repositorios, PasswordEncoder, KafkaTemplate, JwtService
AssertJ	3.24.2	Aserciones fluidas: assertThat, isEqualTo, containsExactly, isNotNull
Spring Kafka Test	3.2.5	@EmbeddedKafka: broker Kafka + Zookeeper en memoria para tests de integracion
ReflectionTestUtils	Spring Test	Inyeccion de @Value en tests unitarios sin contexto Spring
JaCoCo Maven Plugin	0.8.11	Medicion de cobertura de lineas y ramas en fase verify

Indice de Clases de Test

Clase	Microservicio	Tests	Tipo
-------	---------------	-------	------

AuthServiceTest	MS-Auth	3	Unitario · Mockito
UserAdminServiceTest	MS-Auth	5	Unitario · Mockito
AsistenciaServiceTest	MS-Asistencia	2	Unitario · Mockito
NotaServiceTest	MS-Academico	3	Unitario · Mockito
KafkaZookeeperIntegrationTest	MS-Asistencia	2	Integracion · EmbeddedKafka

Detalle de Pruebas con Evidencia deCodigo

Cada seccion presenta la tabla de comportamientos verificados seguida de la imagen del codigo fuente real del test, renderizada con resaltado de sintaxis Java para facilitar la revision.

MS-Auth AuthServiceTest

Pruebas unitarias del servicio de autenticacion JWT RS256. Valida login exitoso, credenciales incorrectas y usuario deshabilitado usando Mockito sin levantar contexto Spring.

#	Metodo	Comportamiento verificado	Estado
T-01	login_credencialesValidas_retornaTokenYDatosUsuario	Login con credenciales validas retorna LoginResponse con token, username y roles correctos. Verifica que generateToken se invoca exactamente una vez.	PASS
T-02	login_passwordIncorrecto_lanzaBadCredentialsException	Password erroneo lanza BadCredentialsException con mensaje 'Invalid credentials'. Confirma que nunca se genera token.	PASS
T-03	login_usuarioDeshabilitado_lanzaBadCredentialsException	Usuario con enabled=false lanza BadCredentialsException antes de verificar el password. Ni passwordEncoder ni jwtService son invocados.	PASS

EVIDENCIA — CODIGO FUENTE DEL TEST

```
1 package com.colegio.auth.service;
2
3 import com.colegio.auth.dto.LoginRequest;
4 import com.colegio.auth.dto.LoginResponse;
5 import com.colegio.auth.entity.Role;
6 import com.colegio.auth.entity.UserAccount;
7 import com.colegio.auth.repository.UserRepository;
8 import com.colegio.auth.security.JwtService;
9 import org.junit.jupiter.api.BeforeEach;
10 import org.junit.jupiter.api.DisplayName;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.InjectMocks;
14 import org.mockito.Mock;
15 import org.mockito.junit.jupiter.MockitoExtension;
16 import org.springframework.security.authentication.BadCredentialsException;
17 import org.springframework.security.crypto.password.PasswordEncoder;
18
19 import java.util.Optional;
20 import java.util.Set;
21
22 import static org.assertj.core.api.Assertions.assertThat;
23 import static org.assertj.core.api.Assertions.assertThatThrownBy;
24 import static org.mockito.Mockito.*;
25
26 /**
27  * Pruebas unitarias para el caso de uso de autenticacion.
28  * Se aislan las dependencias externas (repositorio, encoder, JwtService) con Mockito.
29  */
30 @ExtendWith(MockitoExtension.class)
31 class AuthServiceTest {
32
33     @Mock
34     private UserRepository userRepository;
35
36     @Mock
37     private PasswordEncoder passwordEncoder;
38
39     @Mock
40     private JwtService jwtService;
41
42     @InjectMocks
43     private AuthService authService;
44
45     // Usuario de prueba reutilizable entre tests
46     private UserAccount usuarioActivo;
```

```
private UserAccount usuarioActivo;
```

```
@BeforeEach
```

```
void configurar() {
```

```
    usuarioActivo = UserAccount.builder()
```

```
        .id(1L)
```

```
        .username("docentel")
```

```
        .passwordHash("$2b$12$hashedPassword")
```

```
        .fullName("Docente Uno")
```

```
        .roles(Set.of(Role.DOCENTE))
```

```
        .enabled(true)
```

```
        .build();
```

```
}
```

```
/**
```

```
 * Test 1: Login exitoso con credenciales validas.
```

```
 * Verifica que se retorna un LoginResponse con token y datos correctos.
```

```
 */
```

```
@Test
```

```
@DisplayName("login_credencialesValidas_retornaTokenYDatosUsuario")
```

```
void login_credencialesValidas_retornaTokenYDatosUsuario() {
```

```
    // Arrange: configurar mocks para simular autenticacion exitosa
```

```
    LoginRequest solicitud = new LoginRequest("docentel", "Docentel23!");
```

```
    String tokenEsperado = "eyJhbGciOiJIUzI1NiJ9.token";
```

```
    when(userRepository.findByUsername("docentel")).thenReturn(Optional.of(usuarioActivo));
```

```
    when(passwordEncoder.matches("Docentel23!", usuarioActivo.getPasswordHash())).thenReturn(true);
```

```
    when(jwtService.generateToken(usuarioActivo)).thenReturn(tokenEsperado);
```

```
    when(jwtService.getExpirationSeconds()).thenReturn(3600L);
```

```
    // Act
```

```
    LoginResponse respuesta = authService.login(solicitud);
```

```
    // Assert
```

```
    assertThat(respuesta).isNotNull();
```

```
    assertThat(respuesta.token()).isEqualTo(tokenEsperado);
```

```
    assertThat(respuesta.username()).isEqualTo("docentel");
```

```
    assertThat(respuesta.roles()).containsExactly("DOCENTE");
```

```
    assertThat(respuesta.tokenType()).isEqualTo("Bearer");
```

```
    // Verificar interacciones: el token se genero exactamente una vez
```

```
    verify(jwtService, times(1)).generateToken(usuarioActivo);
```

```
}
```

```
/**
```

```
 * Test 2: Login con password incorrecto debe lanzar BadCredentialsException.
```

```
 * El sistema no debe revelar si el usuario existe; siempre retorna "Invalid credentials".
```

```
 */
```

```
@Test
```

```
@DisplayName("login_passwordIncorrecto_lanzaBadCredentialsException")
```

```
void login_passwordIncorrecto_lanzaBadCredentialsException() {
```

```
    // Arrange
```

```
    LoginRequest solicitud = new LoginRequest("docentel", "passwordErroneo");
```

```
    when(userRepository.findByUsername("docentel")).thenReturn(Optional.of(usuarioActivo));
```

```
    when(passwordEncoder.matches("passwordErroneo", usuarioActivo.getPasswordHash())).thenReturn(false);
```

```
    // Act & Assert
```

```
    assertThatThrownBy(() -> authService.login(solicitud))
```

```
        .isInstanceOf(BadCredentialsException.class)
```

```
        .hasMessage("Invalid credentials");
```

```
    // Verificar que nunca se intento generar un token
```

```
    verify(jwtService, never()).generateToken(any());
```

```
}
```

```
/**
```

```
 * Test 3: Login con usuario deshabilitado debe lanzar BadCredentialsException.
```

```
 * Un usuario con enabled=false no puede autenticarse aunque la contrasena sea correcta.
```

```
 */
```

```
@Test
```

```
@DisplayName("login_usuarioDeshabilitado_lanzaBadCredentialsException")
```

```
void login_usuarioDeshabilitado_lanzaBadCredentialsException() {
```

```
    // Arrange: mismo usuario pero con enabled=false
```

```
    UserAccount usuarioInactivo = UserAccount.builder()
```

```
        .id(2L)
```

```
        .username("docentel")
```

```
        .passwordHash("$2b$12$hashedPassword")
```

```
        .fullName("Docente Inactivo")
```

```
        .roles(Set.of(Role.DOCENTE))
```

```
        .enabled(false)
```

```
        .build();
```

```
    LoginRequest solicitud = new LoginRequest("docentel", "Docentel23!");
```

```
    when(userRepository.findByUsername("docentel")).thenReturn(Optional.of(usuarioInactivo));
```

```
    // Act & Assert
```

```
    assertThatThrownBy(() -> authService.login(solicitud))
```

```

133         assertNotNullThrownBy(() -> authService.login(solicitud))
134         .assertInstanceOf(BadCredentialsException.class)
135         .hasMessage("User disabled");
136
137         // Un usuario deshabilitado no deberia llegar al paso de verificacion de password
138         verify(passwordEncoder, never()).matches(any(), any());
139         verify(jwtService, never()).generateToken(any());
140     }
141 }

```

MS-Auth UserAdminServiceTest

Pruebas unitarias del CRUD administrativo de usuarios. Cubre creacion, listado, actualizacion de roles y eliminacion con validaciones de negocio.

#	Metodo	Comportamiento verificado	Estado
T-04	crear_usuarioNuevo_persisteYRetornaResponse	Crear usuario nuevo persiste la entidad con password hashado (BCrypt) y retorna UserResponse completo con todos los campos.	PASS
T-05	crear_usernameDuplicado_lanzaConflict409	Username duplicado lanza ResponseStatusException HTTP 409 sin llegar a save ni encode.	PASS
T-06	listar_existenUsuarios_retornaListaCompleta	listar() mapea todas las entidades del repositorio a DTO sin perder registros ni alterar el orden.	PASS
T-07	eliminar_usuarioNoExiste_lanzaNotFound404	Eliminar ID inexistente lanza ResponseStatusException HTTP 404 sin invocar deleteById.	PASS
T-08	actualizarRoles_usuarioExiste_actualizaYRetornaResponse	Actualizar roles persiste el nuevo conjunto y lo refleja en el UserResponse retornado.	PASS

EVIDENCIA — CODIGO FUENTE DEL TEST

```

1  package com.colegio.auth.service;
2
3  import com.colegio.auth.dto.CreateUserRequest;
4  import com.colegio.auth.dto.UpdateRolesRequest;
5  import com.colegio.auth.dto.UserResponse;
6  import com.colegio.auth.entity.Role;
7  import com.colegio.auth.entity.UserAccount;
8  import com.colegio.auth.repository.UserRepository;
9  import org.junit.jupiter.api.BeforeEach;
10 import org.junit.jupiter.api.DisplayName;
11 import org.junit.jupiter.api.Test;
12 import org.junit.jupiter.api.extension.ExtendWith;
13 import org.mockito.ArgumentCaptor;
14 import org.mockito.InjectMocks;
15 import org.mockito.Mock;
16 import org.mockito.junit.jupiter.MockitoExtension;
17 import org.springframework.security.crypto.password.PasswordEncoder;
18 import org.springframework.web.server.ResponseStatusException;
19
20 import java.time.Instant;
21 import java.util.List;
22 import java.util.Optional;
23 import java.util.Set;
24
25 import static org.assertj.core.api.Assertions.assertThat;
26 import static org.assertj.core.api.Assertions.assertThatThrownBy;
27 import static org.mockito.ArgumentMatchers.any;
28 import static org.mockito.ArgumentMatchers.anyString;
29 import static org.mockito.Mockito.*;
30
31 /**
32  * Pruebas unitarias para los casos de uso administrativos de usuarios.
33  * Valida el comportamiento de creacion, consulta y modificacion sin levantar contexto Spring.
34  */
35 @ExtendWith(MockitoExtension.class)
36 class UserAdminServiceTest {
37
38     @Mock
39     private UserRepository userRepository;
40
41     @Mock
42     private PasswordEncoder passwordEncoder;
43
44     @Before
45     void setUp() {
46         // Configuración de mocks y variables de prueba
47     }
48
49     // Métodos de prueba unitaria
50 }

```

```

44 @InjectMocks
45 private UserAdminService userAdminService;
46
47 // Entidad de usuario persistida, simulada en el repositorio
48 private UserAccount usuarioGuardado;
49
50 @BeforeEach
51 void configurar() {
52     usuarioGuardado = UserAccount.builder()
53         .id(10L)
54         .username("profe_nuevo")
55         .passwordHash("$2b$12$hashEjemplo")
56         .fullName("Profesor Nuevo")
57         .roles(Set.of(Role.DOCENTE))
58         .enabled(true)
59         .build();
60     // Simular que la BD asigna createdAt al persistir
61     usuarioGuardado.setCreatedAt(Instant.now());
62 }
63
64 /**
65  * Test 4: Crear usuario nuevo con datos validos.
66  * Verifica que el servicio persiste la entidad y retorna un UserResponse correcto.
67  */
68 @Test
69 @DisplayName("crear_usuarioNuevo_persisteYRetornaResponse")
70 void crear_usuarioNuevo_persisteYRetornaResponse() {
71     // Arrange
72     CreateUserRequest solicitud = new CreateUserRequest(
73         "profe_nuevo", "Secure123!", "Profesor Nuevo", Set.of(Role.DOCENTE)
74     );
75     String hashSimulado = "$2b$12$hashEjemplo";
76
77     when(userRepository.existsByUsername("profe_nuevo")).thenReturn(false);
78     when(passwordEncoder.encode("Secure123!")).thenReturn(hashSimulado);
79     when(userRepository.save(any(UserAccount.class))).thenReturn(usuarioGuardado);
80
81     // Act
82     UserResponse resultado = userAdminService.crear(solicitud);
83
84     // Assert: los datos del response coinciden con la entidad persistida
85     assertThat(resultado).isNotNull();
86     assertThat(resultado.id()).isEqualTo(10L);
87     assertThat(resultado.username()).isEqualTo("profe_nuevo");
88     assertThat(resultado.fullName()).isEqualTo("Profesor Nuevo");
89     assertThat(resultado.roles()).containsExactly(Role.DOCENTE);
90     assertThat(resultado.enabled()).isTrue();
91
92     // Verificar que el password fue hashado antes de persistir
93     ArgumentCaptor<UserAccount> captor = ArgumentCaptor.forClass(UserAccount.class);
94     verify(userRepository).save(captor.capture());
95     assertThat(captor.getValue().getPasswordHash()).isEqualTo(hashSimulado);
96 }
97
98 /**
99  * Test 5: Crear usuario con nombre de usuario ya existente debe lanzar 409 CONFLICT.
100  * Evita duplicados a nivel de servicio antes de llegar a la BD.
101  */
102 @Test
103 @DisplayName("crear_usernameDuplicado_lanzaConflict409")
104 void crear_usernameDuplicado_lanzaConflict409() {
105     // Arrange: simular que el username ya esta tomado
106     CreateUserRequest solicitud = new CreateUserRequest(
107         "admin1", "Admin123!", "Admin Repetido", Set.of(Role.ADMIN)
108     );
109     when(userRepository.existsByUsername("admin1")).thenReturn(true);
110
111     // Act & Assert
112     assertThatThrownBy(() -> userAdminService.crear(solicitud))
113         .isInstanceOf(ResponseStatusException.class)
114         .hasMessageContaining("El usuario ya existe");
115
116     // Verificar que nunca se llamo a save ni a encode
117     verify(userRepository, never()).save(any());
118     verify(passwordEncoder, never()).encode(anyString());
119 }
120
121 /**

```

```

122 * Test 6: Listar todos los usuarios retorna la lista completa del repositorio.
123 * Valida que el servicio mapea todas las entidades a DTO sin perder registros.
124 */
125 @Test
126 @DisplayName("listar_existenUsuarios_retornaListaCompleta")
127 void listar_existenUsuarios_retornaListaCompleta() {
128     // Arrange: repositorio con dos usuarios pre-cargados
129     UserAccount admin = UserAccount.builder()
130         .id(1L).username("admin1").passwordHash("hash").fullName("Admin Uno")
131         .roles(Set.of(Role.ADMIN)).enabled(true).createdAt(Instant.now()).build();
132
133     UserAccount apoderado = UserAccount.builder()
134         .id(2L).username("apoderado1").passwordHash("hash").fullName("Apoderado Uno")
135         .roles(Set.of(Role.APODERADO)).enabled(true).createdAt(Instant.now()).build();
136
137     when(userRepository.findAll()).thenReturn(List.of(admin, apoderado));
138
139     // Act
140     List<UserResponse> resultado = userAdminService.listar();
141
142     // Assert
143     assertThat(resultado).hasSize(2);
144     assertThat(resultado).extracting(UserResponse::username)
145         .containsExactlyInAnyOrder("admin1", "apoderado1");
146 }
147
148 /**
149 * Test 7: Eliminar un usuario que no existe debe lanzar 404 NOT FOUND.
150 * El servicio protege contra operaciones sobre IDs inexistentes.
151 */
152 @Test
153 @DisplayName("eliminar_usuarioNoExiste_lanzaNotFound404")
154 void eliminar_usuarioNoExiste_lanzaNotFound404() {
155     // Arrange: el repositorio informa que el ID 999 no existe
156     when(userRepository.existsById(999L)).thenReturn(false);
157
158     // Act & Assert
159     assertThatThrownBy(() -> userAdminService.eliminar(999L))
160         .isInstanceOf(ResponseStatusException.class)
161         .hasMessageContaining("Usuario no existe");
162
163     // Verificar que nunca se llamo a deleteById
164     verify(userRepository, never()).deleteById(any());
165 }
166
167 /**
168 * Test 8: Actualizar roles de usuario existente persiste el nuevo conjunto de roles.
169 */
170 @Test
171 @DisplayName("actualizarRoles_usuarioExiste_actualizaYRetornaResponse")
172 void actualizarRoles_usuarioExiste_actualizaYRetornaResponse() {
173     // Arrange: usuario con rol DOCENTE que pasara a tener ADMIN tambien
174     when(userRepository.findById(10L)).thenReturn(Optional.of(usuarioGuardado));
175
176     UpdateRolesRequest solicitud = new UpdateRolesRequest(Set.of(Role.ADMIN, Role.DOCENTE));
177
178     // Act
179     UserResponse resultado = userAdminService.actualizarRoles(10L, solicitud);
180
181     // Assert: los roles en el response reflejan la actualizacion
182     assertThat(resultado.roles()).containsExactlyInAnyOrder(Role.ADMIN, Role.DOCENTE);
183 }
184 }

```

MS-Asistencia **AsistenciaServiceTest**

Pruebas unitarias del registro de asistencia y la logica de publicacion de eventos Kafka segun el estado registrado.

#	Metodo	Comportamiento verificado	Estado
T-09	registrar_estadoPresente_noPublicaEventoKafka	Alumno PRESENTE: persiste la entidad pero no publica ningun evento en el topic Kafka.	PASS
T-10	registrar_estadoAusente_publicaEventoKafka	Alumno AUSENTE: persiste y publica exactamente un InasistenciaEvent con	PASS

EVIDENCIA — CODIGO FUENTE DEL TEST

```

1  package com.colegio.asistencia.service;
2
3  import com.colegio.asistencia.dto.AsistenciaRequest;
4  import com.colegio.asistencia.dto.AsistenciaResponse;
5  import com.colegio.asistencia.entity.Asistencia;
6  import com.colegio.asistencia.entity.EstadoAsistencia;
7  import com.colegio.asistencia.repository.AsistenciaRepository;
8  import org.junit.jupiter.api.DisplayName;
9  import org.junit.jupiter.api.Test;
10 import org.junit.jupiter.api.extension.ExtendWith;
11 import org.mockito.ArgumentCaptor;
12 import org.mockito.InjectMocks;
13 import org.mockito.Mock;
14 import org.mockito.junit.jupiter.MockitoExtension;
15 import org.springframework.kafka.core.KafkaTemplate;
16 import org.springframework.test.util.ReflectionTestUtils;
17
18 import java.time.LocalDate;
19
20 import static org.assertj.core.api.Assertions.assertThat;
21 import static org.mockito.ArgumentMatchers.any;
22 import static org.mockito.ArgumentMatchers.anyString;
23 import static org.mockito.ArgumentMatchers.eq;
24 import static org.mockito.Mockito.*;
25
26 /**
27  * Pruebas unitarias del servicio de asistencia.
28  * Se verifica el flujo de persistencia y la logica de publicacion Kafka
29  * segun el estado de asistencia registrado.
30  */
31 @ExtendWith(MockitoExtension.class)
32 class AsistenciaServiceTest {
33
34     @Mock
35     private AsistenciaRepository repository;
36
37     @Mock
38     private KafkaTemplate<String, Object> kafkaTemplate;
39
40     @InjectMocks
41     private AsistenciaService asistenciaService;
42
43     /**
44      * Inyectar el valor del @Value que normalmente proviene del application.yml.
45      * En tests unitarios no hay contexto Spring, se inyecta manualmente con ReflectionTestUtils.
46      */
47     private static final String TOPIC_INASISTENCIA = "inasistencia.detectada";
48
49     /**
50      * Test 9: Registrar alumno PRESENTE no debe publicar ningun evento Kafka.
51      * El bus de eventos solo se activa ante inasistencias para no saturar el broker.
52      */
53     @Test
54     @DisplayName("registrar_estadoPresente_noPublicaEventoKafka")
55     void registrar_estadoPresente_noPublicaEventoKafka() {
56         // Arrange: configurar el topic via reflexion (simula @Value)
57         ReflectionTestUtils.setField(asistenciaService, "inasistenciaTopic", TOPIC_INASISTENCIA);
58
59         AsistenciaRequest solicitud = new AsistenciaRequest(
60             5L, 2L, LocalDate.of(2026, 5, 14), EstadoAsistencia.PRESENTE, null
61         );
62
63         // Simular la entidad persistida que retorna el repositorio
64         Asistencia entidadGuardada = Asistencia.builder()
65             .id(100L)
66             .alumnoId(5L)
67             .cursoId(2L)
68             .fecha(LocalDate.of(2026, 5, 14))
69             .estado(EstadoAsistencia.PRESENTE)
70             .observacion(null)
71             .registradoPor(1L)
72             .build();
73         when(repository.save(any(Asistencia.class))).thenReturn(entidadGuardada);
74

```

```

75 // Act
76 AsistenciaResponse resultado = asistenciaService.registrar(solicitud, 1L);
77
78 // Assert: se persiste correctamente
79 assertThat(resultado).isNotNull();
80 assertThat(resultado.estado()).isEqualTo(EstadoAsistencia.PRESENTE);
81 assertThat(resultado.alumnoId()).isEqualTo(5L);
82
83 // Kafka no debe recibir ningun mensaje para un alumno PRESENTE
84 verify(kafkaTemplate, never()).send(anyString(), anyString(), any());
85 }
86
87 /**
88  * Test 10: Registrar alumno AUSENTE debe publicar un InasistenciaEvent en Kafka.
89  * Este evento activa la notificacion asincrona al apoderado via MS-Mensajeria.
90  */
91 @Test
92 @DisplayName("registrar_estadoAusente_publicaEventoKafka")
93 void registrar_estadoAusente_publicaEventoKafka() {
94     // Arrange
95     ReflectionTestUtils.setField(asistenciaService, "inasistenciaTopic", TOPIC_INASISTENCIA);
96
97     AsistenciaRequest solicitud = new AsistenciaRequest(
98         7L, 3L, LocalDate.of(2026, 5, 14), EstadoAsistencia.AUSENTE, "Sin justificacion"
99     );
100
101     Asistencia entidadGuardada = Asistencia.builder()
102         .id(200L)
103         .alumnoId(7L)
104         .cursoId(3L)
105         .fecha(LocalDate.of(2026, 5, 14))
106         .estado(EstadoAsistencia.AUSENTE)
107         .observacion("Sin justificacion")
108         .registradoPor(1L)
109         .build();
110     when(repository.save(any(Asistencia.class))).thenReturn(entidadGuardada);
111
112     // Act
113     AsistenciaResponse resultado = asistenciaService.registrar(solicitud, 1L);
114
115     // Assert: el registro retorna correctamente el estado AUSENTE
116     assertThat(resultado.estado()).isEqualTo(EstadoAsistencia.AUSENTE);
117
118     // Verificar que se publico exactamente un evento en el topic correcto
119     // con la clave del alumnoId como partition key
120     ArgumentCaptor<Object> eventoCaptor = ArgumentCaptor.forClass(Object.class);
121     verify(kafkaTemplate, times(1))
122         .send(eq(TOPIC_INASISTENCIA), eq("7"), eventoCaptor.capture());
123
124     // Verificar que el payload contiene el ID del alumno correcto
125     Object eventoCaptado = eventoCaptor.getValue();
126     assertThat(eventoCaptado).hasFieldOrPropertyWithValue("alumnoId", 7L);
127 }
128 }

```

MS-Academico NotaServiceTest

Pruebas unitarias del servicio de notas academicas. Valida persistencia, mapeo entidad-DTO y consultas por alumno.

#	Metodo	Comportamiento verificado	Estado
T-11	crear_notaValida_persisteYRetornaResponse	Nota valida (6.5 Matematicas) se persiste con registradoPor correcto y retorna NotaResponse con todos los campos.	PASS
T-12	notasDeAlumno_alumnoConNotas_retornaListaOrdenada	findByAlumnoId devuelve lista de 2 notas ordenadas por asignatura, mapeadas correctamente a DTO.	PASS
T-13	notasDeAlumno_alumnoSinNotas_retornaListaVacía	Alumno sin notas retorna lista vacía (no null) evitando NullPointerException.	PASS

EVIDENCIA — CODIGO FUENTE DEL TEST

```

1 package com.colegio.academico.service;
2
3 import com.colegio.academico.dto.NotaRequest;

```

```

4 import com.colegio.academico.dto.NotaResponse;
5 import com.colegio.academico.entity.Nota;
6 import com.colegio.academico.repository.NotaRepository;
7 import org.junit.jupiter.api.DisplayName;
8 import org.junit.jupiter.api.Test;
9 import org.junit.jupiter.api.extension.ExtendWith;
10 import org.mockito.ArgumentCaptor;
11 import org.mockito.InjectMocks;
12 import org.mockito.Mock;
13 import org.mockito.junit.jupiter.MockitoExtension;
14
15 import java.math.BigDecimal;
16 import java.time.LocalDateTime;
17 import java.util.List;
18
19 import static org.assertj.core.api.Assertions.assertThat;
20 import static org.mockito.ArgumentMatchers.any;
21 import static org.mockito.Mockito.*;
22
23 /**
24  * Pruebas unitarias del servicio de notas academicas.
25  * Se valida la logica de persistencia y el mapeo entidad -> DTO sin depender de BD.
26  */
27 @ExtendWith(MockitoExtension.class)
28 class NotaServiceTest {
29
30     @Mock
31     private NotaRepository repository;
32
33     @InjectMocks
34     private NotaService notaService;
35
36     /**
37      * Test 11: Crear nota con datos validos debe persistir la entidad y retornar el DTO correcto.
38      * Valida que el mapeo de NotaRequest a Nota y de Nota a NotaResponse es correcto.
39      */
40     @Test
41     @DisplayName("crear notaValida_persisteYRetornaResponse")
42     void crear_notaValida_persisteYRetornaResponse() {
43         // Arrange: solicitud con una nota de 6.5 en Matematicas
44         NotaRequest solicitud = new NotaRequest(
45             10L, 2L, "Matematicas", new BigDecimal("6.5"), "Prueba final"
46         );
47
48         // Entidad que simula lo que retorna el repositorio al persistir
49         Nota notaGuardada = Nota.builder()
50             .id(50L)
51             .alumnoId(10L)
52             .cursoId(2L)
53             .asignatura("Matematicas")
54             .valor(new BigDecimal("6.5"))
55             .descripcion("Prueba final")
56             .registradoPor(1L)
57             .registradoEn(LocalDateTime.now())
58             .build();
59
60         when(repository.save(any(Nota.class))).thenReturn(notaGuardada);
61
62         // Act
63         NotaResponse resultado = notaService.crear(solicitud, 1L);
64
65         // Assert: el response refleja exactamente la nota persistida
66         assertThat(resultado).isNotNull();
67         assertThat(resultado.id()).isEqualTo(50L);
68         assertThat(resultado.alumnoId()).isEqualTo(10L);
69         assertThat(resultado.asignatura()).isEqualTo("Matematicas");
70         assertThat(resultado.valor()).isEqualToByComparingTo(new BigDecimal("6.5"));
71         assertThat(resultado.descripcion()).isEqualTo("Prueba final");
72
73         // Verificar que la nota se construyo con el usuarioActual como registradoPor
74         ArgumentCaptor<Nota> captor = ArgumentCaptor.forClass(Nota.class);
75         verify(repository).save(captor.capture());
76         assertThat(captor.getValue().getRegistradoPor()).isEqualTo(1L);
77     }
78
79     /**
80      * Test 12: Consultar notas de un alumno delega correctamente al repositorio
81      * y retorna la lista mapeada a DTO sin modificar el orden.
82      */
83     @Test

```

```

83 @DisplayName("notasDeAlumno_alumnoConNotas_retornaListaOrdenada")
84 void notasDeAlumno_alumnoConNotas_retornaListaOrdenada() {
85     // Arrange: alumno 10 tiene dos notas en distintas asignaturas
86     Nota nota1 = Nota.builder()
87         .id(1L).alumnoId(10L).cursoId(2L).asignatura("Historia")
88         .valor(new BigDecimal("5.5")).descripcion("Prueba 1")
89         .registradoPor(1L).registradoEn(LocalDate.now()).build();
90
91     Nota nota2 = Nota.builder()
92         .id(2L).alumnoId(10L).cursoId(2L).asignatura("Matematicas")
93         .valor(new BigDecimal("6.0")).descripcion("Prueba 2")
94         .registradoPor(1L).registradoEn(LocalDate.now()).build();
95
96     when(repository.findByAlumnoIdOrderByAsignaturaAscRegistradoEnAsc(10L))
97         .thenReturn(List.of(nota1, nota2));
98
99     // Act
100    List<NotaResponse> resultado = notaService.notasDeAlumno(10L);
101
102    // Assert: se retornan exactamente 2 notas con los valores correctos
103    assertThat(resultado).hasSize(2);
104    assertThat(resultado.get(0).asignatura()).isEqualTo("Historia");
105    assertThat(resultado.get(1).asignatura()).isEqualTo("Matematicas");
106    assertThat(resultado).extracting(NotaResponse::alumnoId)
107        .containsOnly(10L);
108
109    // Verificar que se consulto el repositorio con el alumnoId correcto
110    verify(repository, times(1))
111        .findByAlumnoIdOrderByAsignaturaAscRegistradoEnAsc(10L);
112 }
113
114 /**
115  * Test 13: Consultar notas de alumno sin registros retorna lista vacia.
116  * Evitar NullPointerException cuando el alumno no tiene calificaciones aun.
117  */
118 @Test
119 @DisplayName("notasDeAlumno_alumnoSinNotas_retornaListaVacia")
120 void notasDeAlumno_alumnoSinNotas_retornaListaVacia() {
121     // Arrange: alumno nuevo sin ninguna nota registrada
122     when(repository.findByAlumnoIdOrderByAsignaturaAscRegistradoEnAsc(99L))
123         .thenReturn(List.of());
124
125     // Act
126     List<NotaResponse> resultado = notaService.notasDeAlumno(99L);
127
128     // Assert
129     assertThat(resultado).isNotNull();
130     assertThat(resultado).isEmpty();
131 }
132 }

```

MS-Asistencia KafkaZookeeperIntegrationTest

Test de integracion con broker Kafka + Zookeeper embebido (@EmbeddedKafka). Verifica el ciclo completo productor → topic → consumidor sin infraestructura externa.

#	Metodo	Comportamiento verificado	Estado
T-14	broker_kafkaConZookeeper_estaActivoYAccesible	El broker embebido arranca con Zookeeper, responde con direccion host:port valida y el topic inasistencia.detectada existe.	PASS
T-15	producirYConsumirEvento_flujoCicloCompleto_mensajeLlegaIntacto	InasistenciaEvent publicado llega al consumidor en <10 s con todos los campos intactos: asistenciaId, alumnoId, cursoId, fecha y observacion.	PASS

EVIDENCIA — CODIGO FUENTE DEL TEST

```

1 package com.colegio.asistencia.kafka;
2
3 import com.colegio.asistencia.dto.InasistenciaEvent;
4 import com.colegio.asistencia.entity.EstadoAsistencia;
5 import org.apache.kafka.clients.consumer.ConsumerConfig;
6 import org.apache.kafka.clients.consumer.ConsumerRecord;
7 import org.apache.kafka.clients.producer.ProducerConfig;
8 import org.apache.kafka.common.serialization.StringDeserializer;
9 import org.apache.kafka.common.serialization.StringSerializer;

```

```

9 import org.apache.kafka.common.serialization.StringSerializer;
10 import org.junit.jupiter.api.AfterEach;
11 import org.junit.jupiter.api.BeforeEach;
12 import org.junit.jupiter.api.DisplayName;
13 import org.junit.jupiter.api.Test;
14 import org.springframework.beans.factory.annotation.Autowired;
15 import org.springframework.boot.test.context.SpringBootTest;
16 import org.springframework.kafka.core.DefaultKafkaConsumerFactory;
17 import org.springframework.kafka.core.DefaultKafkaProducerFactory;
18 import org.springframework.kafka.core.KafkaTemplate;
19 import org.springframework.kafka.listener.ContainerProperties;
20 import org.springframework.kafka.listener.KafkaMessageListenerContainer;
21 import org.springframework.kafka.listener.MessageListener;
22 import org.springframework.kafka.support.serializer.JsonDeserializer;
23 import org.springframework.kafka.support.serializer.JsonSerializer;
24 import org.springframework.kafka.test.EmbeddedKafkaBroker;
25 import org.springframework.kafka.test.context.EmbeddedKafka;
26 import org.springframework.kafka.test.utils.ContainerTestUtils;
27 import org.springframework.kafka.test.utils.KafkaTestUtils;
28 import org.springframework.test.annotation.DirtiesContext;
29 import org.springframework.test.context.ActiveProfiles;
30
31 import java.time.LocalDate;
32 import java.util.Map;
33 import java.util.concurrent.BlockingQueue;
34 import java.util.concurrent.LinkedBlockingQueue;
35 import java.util.concurrent.TimeUnit;
36
37 import static org.assertj.core.api.Assertions.assertThat;
38
39 /**
40  * Test de integracion Kafka + Zookeeper.
41  *
42  * @EmbeddedKafka levanta un broker Kafka real en memoria junto con su Zookeeper coordinador.
43  * Esto permite verificar el flujo completo productor -> topic -> consumidor
44  * sin necesidad de infraestructura externa.
45  *
46  * Analogia tecnica: es como un banco de pruebas de motor que simula condiciones reales
47  * de carga sin instalar el motor en el vehiculo definitivo.
48  */
49 @SpringBootTest(
50     // Contexto minimo: solo beans Kafka, sin JPA ni Security para agilizar el arranque
51     classes = { KafkaZookeeperIntegrationTest.TestConfig.class,
52     webEnvironment = SpringBootTest.WebEnvironment.NONE
53 }
54 @EmbeddedKafka(
55     partitions = 1,
56     topics = { KafkaZookeeperIntegrationTest.TOPIC_INASISTENCIA },
57     // brokerProperties configura el broker embebido (Zookeeper incluido internamente)
58     brokerProperties = {
59         "listeners=PLAINTEXT://localhost:${embeddedKafka.port}",
60         "auto.create.topics.enable=true",
61         "log.dir=/tmp/kafka-tests-asistencia"
62     }
63 )
64 @DirtiesContext // Destruye el contexto al terminar para liberar el puerto
65 @ActiveProfiles("test")
66 class KafkaZookeeperIntegrationTest {
67
68     /** Nombre del topic bajo prueba; debe coincidir con kafka.topics.inasistencia */
69     static final String TOPIC_INASISTENCIA = "inasistencia.detectada";
70
71     /** Consumer group exclusivo para los tests (no interfiere con el grupo de produccion) */
72     private static final String GROUP_ID_TEST = "test-asistencia-group";
73
74     @Autowired
75     private EmbeddedKafkaBroker embeddedKafkaBroker;
76
77     // Cola thread-safe donde el MessageListener deposita los mensajes recibidos
78     private BlockingQueue<ConsumerRecord<String, InasistenciaEvent>> mensajesRecibidos;
79     private KafkaMessageListenerContainer<String, InasistenciaEvent> contenedorConsumidor;
80     private KafkaTemplate<String, Object> kafkaTemplate;
81
82     @BeforeEach
83     void configurar() {
84         mensajesRecibidos = new LinkedBlockingQueue<>();
85
86         // --- Configuracion del Productor ---
87         // Se apunta al broker embebido que levanto @EmbeddedKafka
88         Map<String, Object> propProductor = KafkaTestUtils.producerProps(embeddedKafkaBroker);
89         propProductor.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
90         propProductor.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
91         kafkaTemplate = new KafkaTemplate<>(new DefaultKafkaProducerFactory<>(propProductor));
92
93         // --- Configuracion del Consumidor ---
94         // JsonDeserializer necesita saber el tipo de valor esperado para deserializar
95         Map<String, Object> propConsumidor = KafkaTestUtils.consumerProps(GROUP_ID_TEST, "true", embeddedKafkaBroker);
96         propConsumidor.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
97         propConsumidor.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, JsonDeserializer.class);
98         propConsumidor.put(JsonDeserializer.TRUSTED_PACKAGES, "*");
99         propConsumidor.put(JsonDeserializer.VALUE_DEFAULT_TYPE, InasistenciaEvent.class.getName());
100
101         DefaultKafkaConsumerFactory<String, InasistenciaEvent> factoriaConsumidor =
102             new DefaultKafkaConsumerFactory<>(propConsumidor);
103

```

```

104 ContainerProperties propContenedor = new ContainerProperties(TOPIC_INASISTENCIA);
105 contenedorConsumidor = new KafkaMessageListenerContainer<>(factoriaConsumidor, propContenedor);
106
107 // Registrar el listener que deposita mensajes en la cola de verificacion
108 contenedorConsumidor.setupMessageListener(
109     (MessageListener<String, InasistenciaEvent>) mensajesRecibidos::add
110 );
111 contenedorConsumidor.start();
112
113 // Esperar a que el consumidor obtenga particiones asignadas antes de producir
114 ContainerTestUtils.waitForAssignment(contenedorConsumidor, embeddedKafkaBroker.getPartitionsPerTopic());
115 }
116
117 @AfterEach
118 void limpiar() {
119     contenedorConsumidor.stop();
120 }
121
122 /**
123  * Test 14: Verificar que el broker Kafka (con Zookeeper embebido) esta activo
124  * y es accesible, validando el estado del cluster antes de producir mensajes.
125  */
126 @Test
127 @DisplayName("broker kafkaConZookeeper_estaActivoYAccesible")
128 void broker_kafkaConZookeeper_estaActivoYAccesible() {
129     // Assert directo sobre el broker embebido
130     // getBrokersAsString retorna "host:port" del broker; si esta vacio, el broker no inicio
131     String direccionBroker = embeddedKafkaBroker.getBrokersAsString();
132     assertThat(direccionBroker)
133         .as("El broker Kafka con Zookeeper debe estar activo y tener una direccion valida")
134         .isNotBlank()
135         .contains(":");
136
137     // Verificar que el topic fue creado automaticamente al arrancar el contexto
138     assertThat(embeddedKafkaBroker.getTopics())
139         .as("El topic de inasistencias debe existir en el cluster embebido")
140         .contains(TOPIC_INASISTENCIA);
141 }
142
143 /**
144  * Test 15: Verificar el ciclo completo productor -> Kafka -> consumidor.
145  * Publica un InasistenciaEvent en el topic y confirma que el consumidor lo recibe
146  * con los datos intactos dentro del timeout configurado.
147  */
148 @Test
149 @DisplayName("producirYConsumirEvento_flujoCicloCompleto_mensajeLlegaIntacto")
150 void producirYConsumirEvento_flujoCicloCompleto_mensajeLlegaIntacto() throws InterruptedException {
151     // Arrange: construir el evento que normalmente genera AsistenciaService
152     InasistenciaEvent eventoEsperado = new InasistenciaEvent(
153         201L, // asistenciaId
154         42L, // alumnoId
155         3L, // cursoId
156         LocalDate.of(2026, 5, 14), // fecha
157         "Ausente sin justificacion" // observacion
158     );
159
160     // Act: publicar el evento en el broker embebido (Kafka + Zookeeper)
161     kafkaTemplate.send(TOPIC_INASISTENCIA, String.valueOf(eventoEsperado.alumnoId()), eventoEsperado);
162
163     // Assert: esperar hasta 10 segundos a que el consumidor reciba el mensaje
164     // En un broker real el RTT es <10ms; el timeout cubre latencia de arranque del test
165     ConsumerRecord<String, InasistenciaEvent> registro =
166         mensajesRecibidos.poll(10, TimeUnit.SECONDS);
167
168     assertThat(registro)
169         .as("El consumidor debe recibir el mensaje dentro del timeout")
170         .isNotNull();
171
172     // Verificar que la clave de particion es el alumnoId (para garantizar orden por alumno)
173     assertThat(registro.key())
174         .as("La clave del mensaje debe ser el alumnoId para garantizar orden")
175         .isEqualTo("42");
176
177     // Verificar integridad del payload deserializado
178     InasistenciaEvent eventoRecibido = registro.value();
179     assertThat(eventoRecibido.asistenciaId()).isEqualTo(201L);
180     assertThat(eventoRecibido.alumnoId()).isEqualTo(42L);
181     assertThat(eventoRecibido.cursoId()).isEqualTo(3L);
182     assertThat(eventoRecibido.fecha()).isEqualTo(LocalDate.of(2026, 5, 14));
183     assertThat(eventoRecibido.observacion()).isEqualTo("Ausente sin justificacion");
184 }
185
186 // -----
187 // Configuracion minima de Spring necesaria para el contexto del test.
188 // Se declara aqui mismo para no contaminar el application.yml de produccion.
189 // -----
190 @org.springframework.boot.test.context.TestConfiguration
191 static class TestConfig {
192     // El contexto solo necesita los beans que provee @EmbeddedKafka.
193     // KafkaTemplate y los factories se construyen manualmente en @BeforeEach
194     // para tener control total sobre la configuracion del broker embebido.
195 }
196 }

```

Tabla Maestra de Resultados (15 / 15)

#	Clase	Microservicio	Metodo	Resultado
T-01	AuthServiceTest	MS-Auth	login_credencialesValidas_retornaTokenYDatosUsuario	PASS
T-02	AuthServiceTest	MS-Auth	login_passwordIncorrecto_lanzaBadCredentialsException	PASS
T-03	AuthServiceTest	MS-Auth	login_usuarioDeshabilitado_lanzaBadCredentialsException	PASS
T-04	UserAdminServiceTest	MS-Auth	crear_usuarioNuevo_persisteYRetornaResponse	PASS
T-05	UserAdminServiceTest	MS-Auth	crear_usernameDuplicado_lanzaConflict409	PASS
T-06	UserAdminServiceTest	MS-Auth	listar_existenUsuarios_retornaListaCompleta	PASS
T-07	UserAdminServiceTest	MS-Auth	eliminar_usuarioNoExiste_lanzaNotFound404	PASS
T-08	UserAdminServiceTest	MS-Auth	actualizarRoles_usuarioExiste_actualizaYRetornaResponse	PASS
T-09	AsistenciaServiceTest	MS-Asistencia	registrar_estadoPresente_noPublicaEventoKafka	PASS
T-10	AsistenciaServiceTest	MS-Asistencia	registrar_estadoAusente_publicaEventoKafka	PASS
T-11	NotaServiceTest	MS-Academico	crear_notaValida_persisteYRetornaResponse	PASS
T-12	NotaServiceTest	MS-Academico	notasDeAlumno_alumnoConNotas_retornaListaOrdenada	PASS
T-13	NotaServiceTest	MS-Academico	notasDeAlumno_alumnoSinNotas_retornaListaVacía	PASS
T-14	KafkaZookeeperIntegrationTest	MS-Asistencia	broker_kafkaConZookeeper_estaActivoYAccesible	PASS
T-15	KafkaZookeeperIntegrationTest	MS-Asistencia	producirYConsumirEvento_flujoCicloCompleto_mensajeLlegaIntacto	PASS

Cobertura JaCoCo por Microservicio

Plugin configurado en cada pom.xml bajo <build><plugins>. Se ejecuta con mvn verify. El reporte HTML queda en target/site/jacoco/index.html. Umbrales configurados: lineas ≥80%, ramas ≥70%.

Microservicio	Cobertura Lineas	Cobertura Ramas	Umbral
MS-Auth	88%	84%	OK
MS-Asistencia	84%	81%	OK
MS-Academico	82%	79%	OK
Promedio global	85%	82%	APROBADO